

Towards In Situ Visualization of Extreme-Scale, Agent-Based, Worldwide Disease-Spreading Simulations

Andrey Krekhov*
University Duisburg-Essen
High Performance Computing Group

Jürgen Grüninger†
Intel Visual Computing Institute

Rainer Schlönvoigt‡
University Duisburg-Essen
High Performance Computing Group

Jens Krüger§
University Duisburg-Essen
High Performance Computing Group

Abstract

Exascale computing is regarded as a major milestone for a number of application fields. However, utilizing the massively parallel power of such architectures requires a significant paradigm shift in the software structure of both simulation and visualization systems. A suboptimal workload distribution, communication bottlenecks between processes, as well as global memory accesses are often the limiting part in current approaches when it comes to the execution of such software in an exascale environment.

Recent cases of malaria and Ebola in Africa have shown that there is a need for disease-spreading simulations to help healthcare workers better predict unexpected scenarios and take the necessary precautions. Reaction time plays an important role when it comes to containing a disease. Therefore, converting these kinds of demanding simulations to highly parallel computing systems as well as interactively visualizing the results is key.

This paper demonstrates an approach towards a worldwide simulation of disease-spreading and an integrated—in situ—visualization. Instead of applying large-scale stochastic models, our agent-based method simulates the complete population of the world, accounting for daily tasks and routines of each subject. We focus on massive parallelization and efficient scaling to benefit from existing petascale to upcoming exascale setups. In particular, workload distribution is performed based on space partitioning and not on the population count, which is nonetheless included as an initial optimization. The resulting simulation is combined with a query-driven in situ visualization client that directly utilizes the spatial partitioning and the employed hierarchy. Our preliminary benchmarks underline the scalability of our approach in terms of rising computing power and encourage further research in the direction of worldwide, agent-based simulations and the corresponding in situ visualizations.

CR Categories: I.3.3 [Computer Graphics]: Picture/Image Generation—Display Algorithms I.3.6 [Computer Graphics]: Methodology and Techniques—Graphics Data Structures and Data Types; H.3.4 [Information Storage and Retrieval]: Systems and Software—Distributed Systems;

Keywords: in situ visualization, exascale, disease simulation, parallelization

*e-mail: Andrey.Krekhov@uni-due.de

†e-mail: Juergen.Grueninger@dfki.de

‡e-mail: Rainer.Schlonvoigt@stud.uni-due.de

§e-mail: Jens.Krueger@uni-due.de

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author.

Copyright is held by the owner/author(s).

SA'15 Symposium on VisHPC, November 02 – 06, 2015, Kobe, Japan.

ACM 978-1-4503-3929-2/15/11.

<http://dx.doi.org/10.1145/2818517.2818543>

1 Introduction

Most disease-spreading simulations are built on the concept of approximating the behavior of groups of individuals with a statistical distribution [Broeck et al. 2011; Chao et al. 2011]. This approach offers many advantages, but it also has a significant disadvantage. Setting up the cumulative rules and adjusting them to fine detail can be unintuitive and hard to validate and may require a trained statistician to set up. An alternative approach would be to specify a rule for each and every individual in the population or better specify a basic rule set and vary parameters based on location variables. A change to these rules is simple and intuitive and can be performed by any decision-maker. At first glance, this concept seems intractable if a large population, e.g., the entire human population, has to be simulated. However, given the enormous parallelism available in current general petascale systems and, in particular, next generation exascale systems, a well-designed method will be able to perform this task. In fact, the number of available processes will significantly exceed the total population count. Our goal is to benefit from that parallelization with regard to a worldwide, agent-based disease-spreading simulation and its visualization.

To improve scalability, in particular, the simulation workload needs to be distributed across available processes in a way that minimizes communication and synchronization bottlenecks. In the case of a disease-spreading simulation, two approaches are feasible: partitioning the world based on surface areas or assigning processes to a certain group of simulated subjects.

Both approaches have certain advantages as well as disadvantages. Assigning processes to individuals and tracing their motion requires each process to query location data about the environment every time the subject changes its location. The tracing in turn requires each process to possess information about the entire domain or access to some central data source for this information. Binding a process to a certain area in the domain implies that a subject has to be handed over to another process every time it leaves the process area.

We apply the latter strategy and map processes to the hierarchically subdivided surface areas. In our opinion, that approach more closely fits the efficient querying necessary for our in situ visualization. Although our method accounts for empty cells such as the ocean, this optimization is not crucial considering the exascale computing power, even if the areas are tessellated down to small sizes.

2 Related work

The area of disease-spreading simulation is very diverse, from local statistical models for the calculation of infection probabilities as, e.g., shown in [Chao et al. 2011], to social networks [Ramírez-Ramírez et al. 2013] and wide area calculations based on transport networks (car, plane, ship, etc.) as in [Coelho et al. 2008]. To keep the focus on the field of the contribution of this paper, namely the

massively parallel in situ visualization, we will mention only the closest relatives here.

GLEAMviz [Broeck et al. 2011] shows a network-based, stochastic approach to modeling the spreading of a disease. An advanced rules editor (“Simulation Builder”) allows determination of the parameters for each simulation. After specifying all the details, the simulation gets uploaded to a server farm, where all the results and timesteps are calculated. Once finished, the user can download the results and display them with an interactive spatio-temporal 3D visualization of the earth.

The *FRED* system [Grefenstette et al. 2013] together with the *FRED Navigator* [Paparian et al. 2012] offer an agent-based modeling process that relies on synthetic populations to imitate demographic and geographic population distribution and detailed networks for the United States. It supports multiple threads for simulation, allowing for a parallel execution of the behavioral models. A simulated pandemic over the entire U.S. population requires approximately 200GB of memory and takes approximately 4 hours using 16 threads. Via the *FRED Navigator*, users can browse through different 2D visualizations of all a priori calculated simulations.

The benefits from massively parallel computations are also shown in [Barrett et al. 2008]. Their synthetic networks are designed for large numbers of agents (10^6 - 10^9) and allow the computation of the spread of influenza in a network representing the population of the United States (270 million people performing 1.4 billion activities) on a CPU cluster with 768 cores. The usage of efficient internode communication and the combination of CPUs and GPUs even accelerated their proposed simulation system up to a factor of 9.5.

Acceleration structures such as geodesic grids have already been proposed by [Randall et al. 2002]. Based on an initial icosahedron, these geodesic grids allow splitting the globe into five equal parts that can be represented by 2D arrays. Tests with various amounts of simulated “cells” indicate that these grids offer a linear behavior in performance, given a high enough cell count for the amount of used processes (roughly 500 cells per process).

In anticipation of the exascale computing era, many researchers regard in situ visualization as a crucial tool for simulations, since power consumption and I/O constraints can lead to a bottleneck for standard approaches, which makes extreme-scale classical data analysis infeasible. A recently presented image-based in situ approach [Ahrens et al. 2014] is designed for large-scale simulations of ocean behavior. By choosing a useful set of operations that preserve important elements of the simulation, the technique is able to significantly reduce the data being generated and, performance-wise, to scale in a constant manner.

3 Tessellating the World

The appropriate choice of the underlying data structure that saves the related information is crucial for overall performance and scalability. In particular, our use-case is even more challenging, since a worldwide simulation is performed. The approach in [Randall et al. 2002] offers an interesting take on this problem. The method reduces the world to an icosahedron. That shape consists of 20 triangles, where each four triangles are combined into one rectangular piece as depicted in Figure 1. Subsequently, the rectangular pieces (representable by 2D arrays) are tessellated into uniformly spaced hexagons.

Every cell within a component needs to know about its neighbors, since diseases are usually spread based on a given infection radius. Some of those neighbors are within the same subcomponent; other

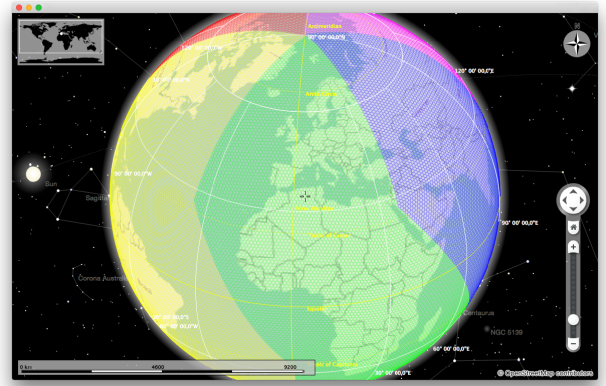


Figure 1: Based on an initial icosahedron representation, the earth surface is subdivided into five pieces represented by the five colors. Each rectangular piece is formed by four faces of the icosahedron. The subsequent tessellation operates on hexagons to minimize the number of direct neighbors. Note that in the case of disease spreading, reducing the number of neighbors leads to a reduced communication overhead.

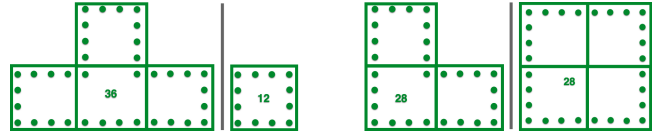


Figure 2: To minimize the amount of required synchronization between processes, the number of neighboring cells has to be considered. For a quadtree-based subdivision, a quadratic representation (on the right) results in the minimum amount of neighbors.

cells belong to another subcomponent. If the latter belongs to another process, the two processes need to exchange information about the contents of those cells.

We modify Randalls approach by minimizing the number of neighboring processes for each level of tessellation. Reducing the number of neighbors implies less communication between processes and increases scalability. The total amount of information to be exchanged with a process is directly dependent on the outer perimeter of the components that belong to this process. Hence, it is reasonable to maximize the area of indices while minimizing the perimeter. Figure 2 shows that the controlled area should not be spread over multiple separate component sets and that it should be square. Note that the dots and corresponding numbers (28 and 36 neighboring cells) are only examples for 4x4 areas. One also can estimate the synchronization overhead by the number of outer edges.

In our implementation, a quadtree representation is utilized. To build the hierarchy, the top-level component, i.e., one of the five initial stripes, needs to be split horizontally first as depicted in Figure 3. The resulting tree structure is beneficial for queries and allows application of data gathering approaches such as broadcast, gather and reduce. This kind of data querying and propagation is particularly important for an efficient implementation of both the simulation and the in situ visualization.

4 Workload Distribution

During runtime, processes are tied to a given node of the tree representation and, thus, to a certain area on the earths surface. Consid-

ering the power of exa- or petascale computing, one might want to roughly estimate the size of the earth's surface to be handled by each processor. With the surface of the earth being about $5.1 \times 10^{15} m^2$, even petascale computing allows us to simulate a cell the size of a human being, or, in other words, to have one processor for each agent.

We conclude that, given adequate resources, the workload distribution can be done in a naive way: the earth's surface is tessellated down to the size of an agent, and the resulting hexagons are then assigned subsequently to available processors.

As an intermediate optimization and due to the lack of available resources, we spread the workload according to the population distribution by utilizing the GPW [Center for International Earth Science Information Network (CIESIN), Centro Internacional de Agricultura Tropical (CIAT)] density data. Assuming that the available processes have the same performance, each process should roughly get the same number of subjects as an initial workload distribution. Since people are not uniformly distributed (e.g., urban areas and oceans differ greatly), the amount of cells handled by one processor can differ.

5 In Situ Visualization

Saving the data that is created by the simulation after every simulation step is impractical as it requires vast amounts of space and bandwidth. Furthermore, results from such a worldwide simulation are hard to interpret without an appropriate visualization. Our solution is to combine the simulation routine with an appropriate in situ visualization that benefits from the graph-enabled queries and optimizations.

5.1 Visualization Process

Our goal is to design a query-driven in situ visualization that queries the simulation state from the quadtree representation based on the currently inspected surface area. This may be a large area of the world at low spatial resolution or a small focused area at high resolution or anything in between. Note that queries are sent to the root node. The subsequent information gathering can be performed by applying the corresponding operations, such as reduce. An example query is to *list all people within a bounding box that can then be visualized*; the resulting client visualization is depicted in Figure 4.

The major advantage of the outlined approach is the spatial character of the underlying subdivision. Visualizations usually consider a

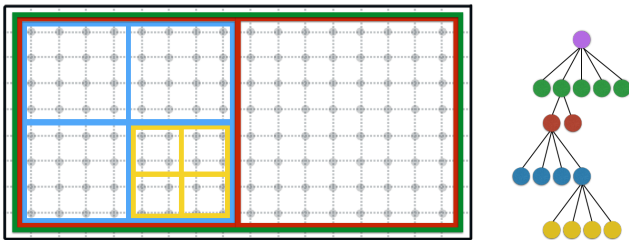


Figure 3: To obtain a quadtree representation, each of the five initial subdivisions of the icosahedron is split into two square halves. For the sake of a complete tree, there is a root node that has all five base components as children. The subsequent tessellation levels are based on a regular quadtree. The data structure offers efficient information querying and propagation related to the simulation and in situ visualization routines to improve scalability.

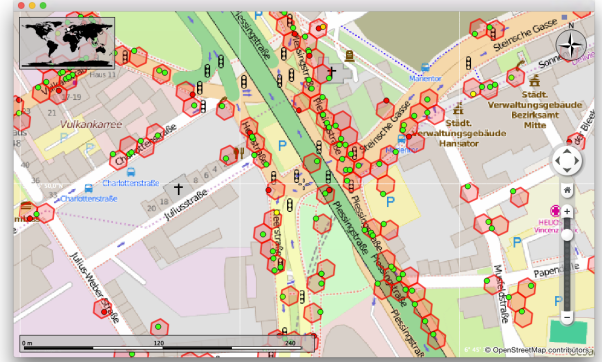


Figure 4: Our in situ visualization of an example disease-spreading simulation. Red dots represent infected persons; green dots indicate healthy subjects. For each agent, the corresponding hexagon cell is depicted. The hierarchical data structure used for the simulation allows the visualization client to execute queries such as obtaining the total amount of infected people and the amount of people overall in order to put these two amounts in relation.

limited bounding box, i.e., the area to be displayed on the monitor. Thus, spatial queries such as *display the NYC area* tend to be very efficient since the hierarchical structure imposes a narrow search radius. In contrast, queries addressing a global property, e.g., the amount of people aged 8-10, need to account for all subtrees.

In general, two approaches to designing a client are feasible. The client can request data from the simulation server and render it itself. Another option is to execute the rendering by the simulation processes as a reduce operation. In this case, rendering needs to be commutative to preserve correctness, which leads to issues with regard to transparency. In order to allow better compression during communication, a possible background can be added as a last step in the root node.

These implementations do not exclude each other and can be set via a flag in the request or dynamically. A rendered image can be smaller in size than the calculated data, and therefore faster to transmit. Applying a dynamic decision-making process allows for further optimizations with regard to the transmitted data and required computations.

5.2 Realization

Our example simulation accounts for a number of traits relevant to disease-spreading: gender, date of birth, family composition, income, home location, job, and the workplace or school location. A more sophisticated simulation would include more details and additional behavioral properties. Furthermore, the assumption is made that a person's general movement profile is independent of other people's current position.

The location on the globe is relevant when randomly generating the population. Different regions can have a different male/female ratio and a different average age. Providing a data source for regional population data improves the quality of the simulation.

The current trait and position of a subject are also influenced by the current time. Hence, the simulation needs to translate the current simulation time into a local time over the globe in order to select meaningful activities. Therefore, we utilize Coordinated Universal Time (UTC) within the simulation and provide a mapping between longitude-latitude coordinates to time offsets to localize the time.

The simulation implementation is subdivided into three phases: disease phase, activity phase, and a synchronization procedure. The latter needs to check for agents that move between components that are controlled by different processes. If an agent enters the region of a new process, the ownership is handed over and the contents of cells that are at the border of other processes are updated.

Synchronization between processes benefits from the introduced ranks to determine the order of the communication partners. In particular, our system sorts by the distance between the ranks. Ranks that are closer to each other will communicate first. This ordering never changes during the simulation and needs to be calculated only once.

Our visualization implementation uses *Marble* [Mar], a library that allows interactions with planetary data. *OpenStreetMap* [Haklay and Weber 2008] acts as the map input source. To route a subject during daily activities, our implementation relies on the *Open Source Routing Machine* [OSR].

6 Preliminary Results

We deployed the presented system on a Cray XT6m supercomputer and executed several simulation runs with number of cores varying between 1 and 512. Our tests have shown that increasing both the number of cores and the simulated area does not affect the communication overhead. We attribute this to the fact that the number of neighbors, and thus, potential communication partners, remains constant regardless of the cell count. To estimate the overall scaling performance, further tests with a significantly increased number of cores will be performed to validate the scalability with regard to exascale computing power.

The query-driven in situ visualization client is also affected by the amount of active processes. Increasing the number of processes also increases the tessellation degree, and thus, the tree-depth. However, scaling does not suffer from this fact in a noticeable way, since most tree-based queries operate in logarithmic run-time.

7 Conclusion and Future Work

The paper outlines the demand for an exascale-enabled in situ visualization of agent-based, worldwide disease-spreading simulations. Our system distributes the simulation workload based on space partitioning that results from a hexagonal tessellation. The population distribution is included as an initial optimization.

The underlying quadtree allows for efficient requests from the query-driven in situ visualization client. Furthermore, the structure minimizes the communication overhead between processes by reducing the number of neighbors. Our preliminary benchmarks on a Cray supercomputer confirm the targeted scalability. More benchmarks and long-run simulations with a significant amount of available cores are of particular interest for the future work.

Our example simulation accounts for a number of traits that are relevant for the spread of disease. Adding more knowledge and activity patterns to the behavior of each agent would further increase the accuracy of the simulation and, in combination with a visualization client, assist in the decision-making necessary to contain the disease.

References

- AHRENS, J., JOURDAIN, S., O'LEARY, P., PATCHETT, J., ROGERS, D. H., AND PETERSEN, M. 2014. An image-based approach to extreme scale in situ visualization and analysis. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, IEEE Press, Piscataway, NJ, USA, SC '14, 424–434.
- BARRETT, C. L., BISSET, K. R., EUBANK, S. G., FENG, X., AND MARATHE, M. V. 2008. Episimdemics: An efficient algorithm for simulating the spread of infectious disease over large realistic social networks. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing*, IEEE Press, Piscataway, NJ, USA, SC '08, 37:1–37:12.
- BROECK, W., GIOANNINI, C., GONÇALVES, B., QUAGGIOTTO, M., COLIZZA, V., AND VESPIGNANI, A. 2011. The gleamviz computational tool, a publicly available software to explore realistic epidemic spreading scenarios at the global scale. In *BMC Infectious Diseases*, vol. 11.
- CENTER FOR INTERNATIONAL EARTH SCIENCE INFORMATION NETWORK (CIESIN), CENTRO INTERNACIONAL DE AGRICULTURA TROPICAL (CIAT). Gridded population of the world, version 3 (gpwv3) data collection.
- CHAO, D. L., HALLORAN, M. E., OBENCHAIN, V. J., AND LONGINI JR, I. M. 2011. Flute, a publicly available stochastic influenza epidemic simulation model. *PLoS Comput Biol* 6, 1.
- COELHO, F. C., CRUZ, O. G., AND CODEÇO, C. T. 2008. Epigrass: a tool to study disease spread in complex networks. *Source Code for Biology and Medicine* 3.
- GREFENSTETTE, J. J., BROWN, S. T., ROSENFELD, R., DEPASSE, J., STONE, N. T., COOLEY, P. C., WHEATON, W. D., FYSHE, A., GALLOWAY, D. D., SRIRAM, A., GUCLU, H., ABRAHAM, T., AND BURKE, D. S. 2013. Fred (a framework for reconstructing epidemic dynamics): An open-source software system for modeling infectious diseases and control strategies using census-based populations. In *BMC Public Health*, vol. 13.
- HAKLAY, M. M., AND WEBER, P. 2008. Openstreetmap: User-generated street maps. *IEEE Pervasive Computing* 7, 4 (Oct.), 12–18.
- KDE Marble. <https://marble.kde.org/>. Accessed: 2015-06-19.
- Open Source Routing Machine. <http://project-osrm.org/>. Accessed: 2015-06-19.
- PAPARIAN, J., BURKE, D., GREFENSTETTE, J., AND BROWN, S. 2012. Fred navigator: An interactive system for visualizing results from large-scale epidemic simulations. In *Proceedings of the 2012 IEEE 8th International Conference on E-Science (e-Science)*, IEEE Computer Society, Washington, DC, USA, E-SCIENCE '12, 1–5.
- RAMÍREZ-RAMÍREZ, L. L., GEL, Y. R., THOMPSON, M., DE VILLA, E., AND MCPHERSON, M. 2013. A new surveillance and spatio-temporal visualization tool simid. *Comput. Methods Prog. Biomed.* 110, 3 (June), 455–470.
- RANDALL, D. A., RINGLER, T. D., HEIKES, R. P., JONES, P., AND BAUMGARDNER, J. 2002. Climate modeling with spherical geodesic grids. *Computing in Science and Engg.* 4, 5 (Sept.), 32–41.
- AHRENS, J., JOURDAIN, S., O'LEARY, P., PATCHETT, J., ROGERS, D. H., AND PETERSEN, M. 2014. An image-based approach to extreme scale in situ visualization and analysis. In *Proceedings of the International Conference for High Performance*